EXPERIMENT NO.6

Name: Niraj Kothawade
Class:D20A
Roll No:30
Batch:B

Aim:Creating Smart Contracts and performing transactions using Solidity and Remix IDE

Theory:

A Smart Contract is a self-executing program stored on the blockchain that automatically enforces the terms of an agreement when predefined conditions are met. It eliminates the need for intermediaries, ensuring transparency, security, and trust between parties. Smart contracts are written in Solidity (for Ethereum-based systems) and executed on the Ethereum Virtual Machine (EVM).

Significance of Smart Contracts in the Ethereum Blockchain:

- **Automation**: Executes actions automatically when conditions are satisfied.
- **Transparency**: The contract code and transactions are visible on the blockchain.
- **Security**: Once deployed, contracts are immutable and tamper-proof.
- **Cost-effective**: Removes intermediaries and reduces transaction costs.
- **Trustless System**: Participants can interact without needing to trust each other directly.

Smart contracts form the backbone of decentralized applications (DApps) and decentralized finance (DeFi) systems on Ethereum.

Implementation:

The project "**Land Ownership Registry System"** focuses on automating land records and ownership transfers using Ethereum-based Smart Contracts.

To ensure the system works, we will implement two primary contracts:

1. **LandRegistry.sol**: Manages the database of lands and their owners.
2. **LandTransfer.sol**: Handles the logic of transferring ownership from a Seller to a Buyer.

Code:-

1. **LandRegistry.sol**

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract LandRegistry {
    struct Land {
        uint id;
        string location;
        uint area;
        address owner;
        bool isRegistered;
    }

    mapping(uint => Land) public lands;
    address public admin;

    constructor() {
        admin = msg.sender;
    }

    function registerLand(uint _id, string memory _location, uint _area, address _owner)
public {
        require(msg.sender == admin, "Only admin can register land");
        require(!lands[_id].isRegistered, "Land already registered");

        lands[_id] = Land(_id, _location, _area, _owner, true);
    }

    function updateOwner(uint _id, address _newOwner) public {
        // This function will be called by the Transfer contract
        lands[_id].owner = _newOwner;
    }

    function getLand(uint _id) public view returns (uint, string memory, uint, address) {
        Land memory l = lands[_id];
```

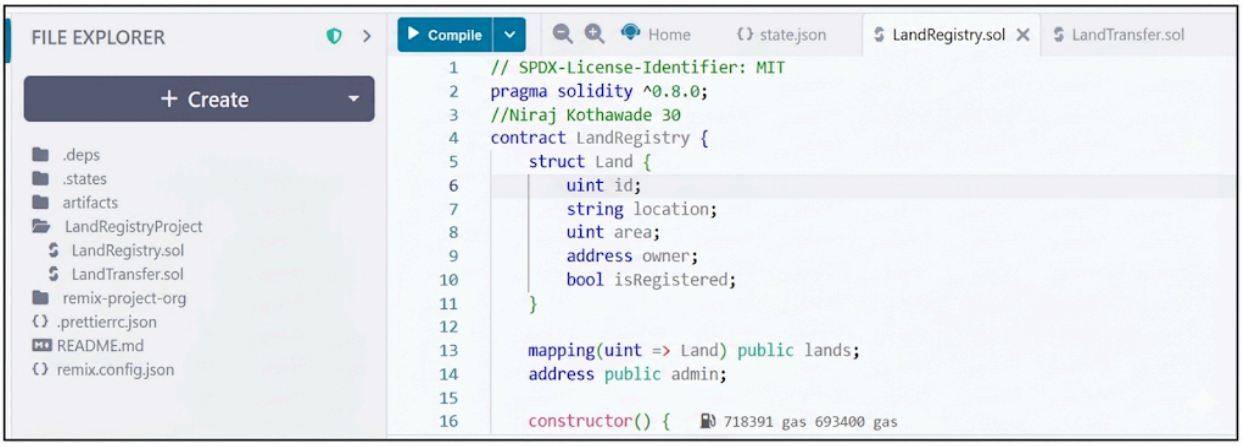
```
        return (l.id, l.location, l.area, l.owner);
    }
}
```

2.**LandTransfer.sol**

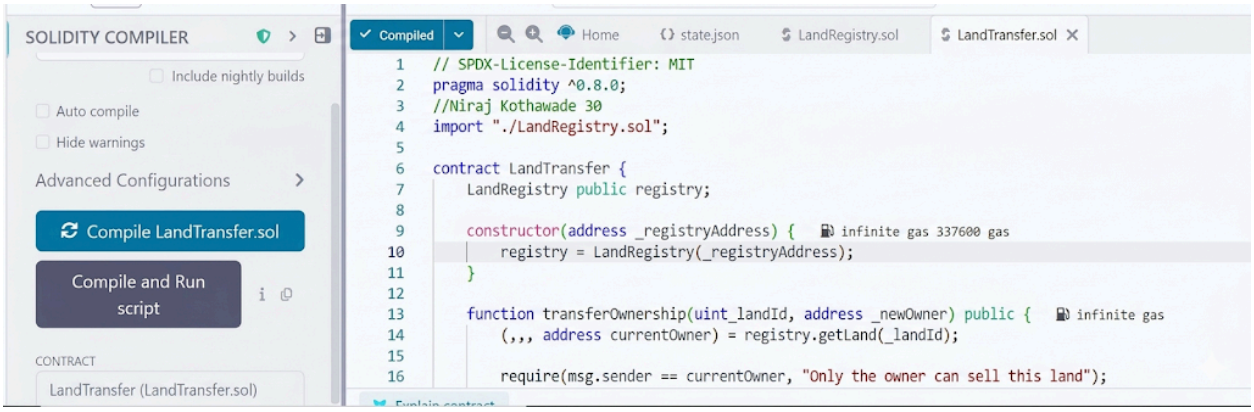
```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "./LandRegistry.sol";

contract LandTransfer {
    LandRegistry public registry;

    constructor(address _registryAddress) {
        registry = LandRegistry(_registryAddress);
    }

    function transferOwnership(uint _landId, address _newOwner) public {
        (,,, address currentOwner) = registry.getLand(_landId);

        require(msg.sender == currentOwner, "Only the owner can sell this land");

        registry.updateOwner(_landId, _newOwner);
    }
}
```

Step 1: Set up the Files

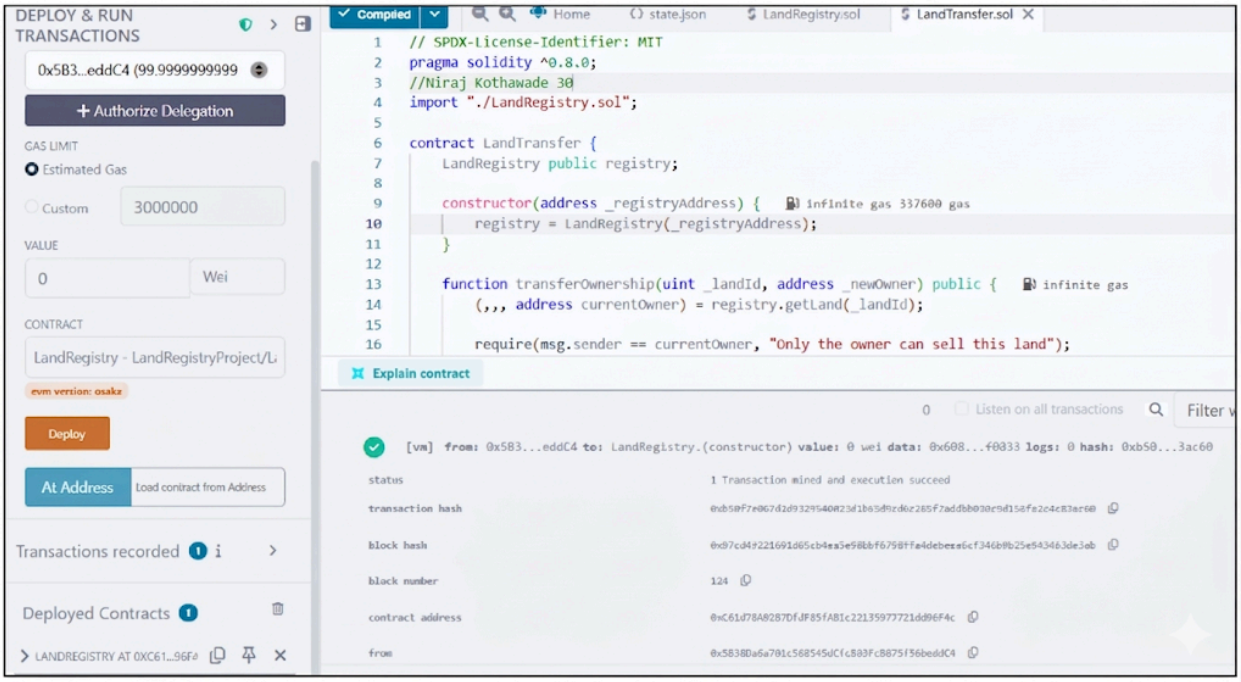
1. Go to Remix IDE.
2. In the File Explorer, create a new folder named LandRegistryProject.
3. Create two files: LandRegistry.sol and LandTransfer.sol.



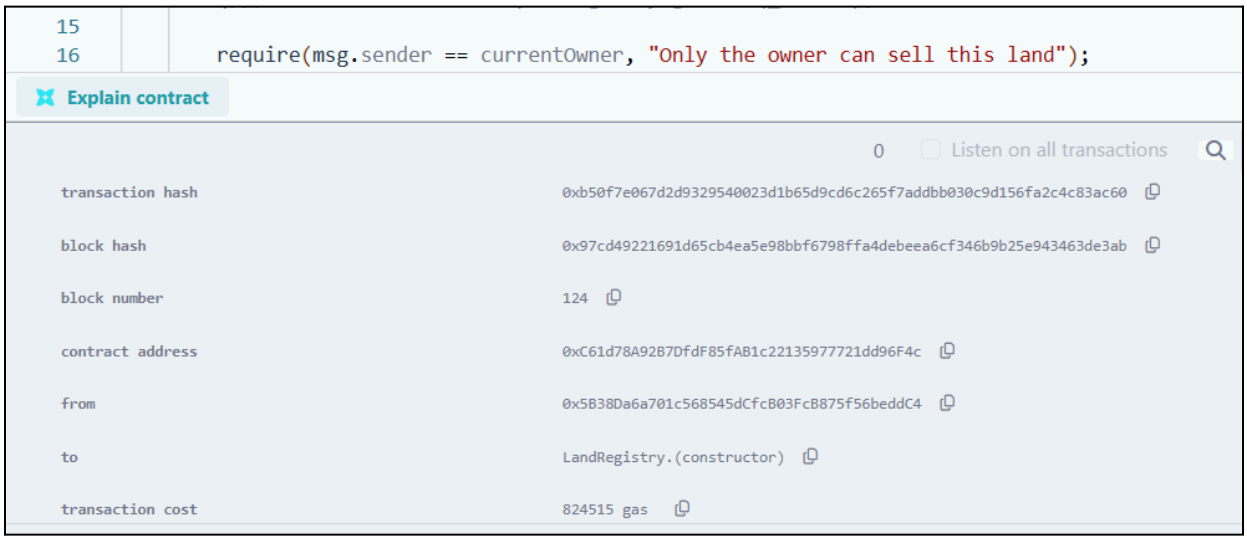
Step 2: Compile both the contracts



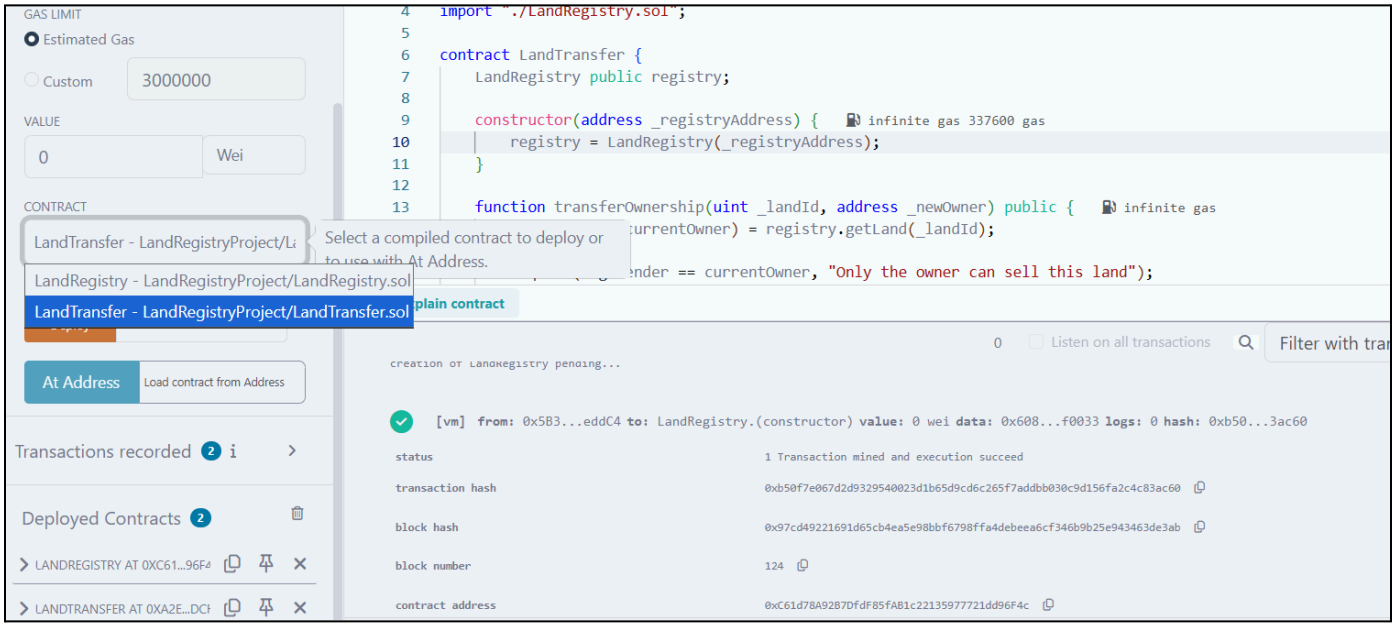
Step 3:Deploy the LandRegistry contract



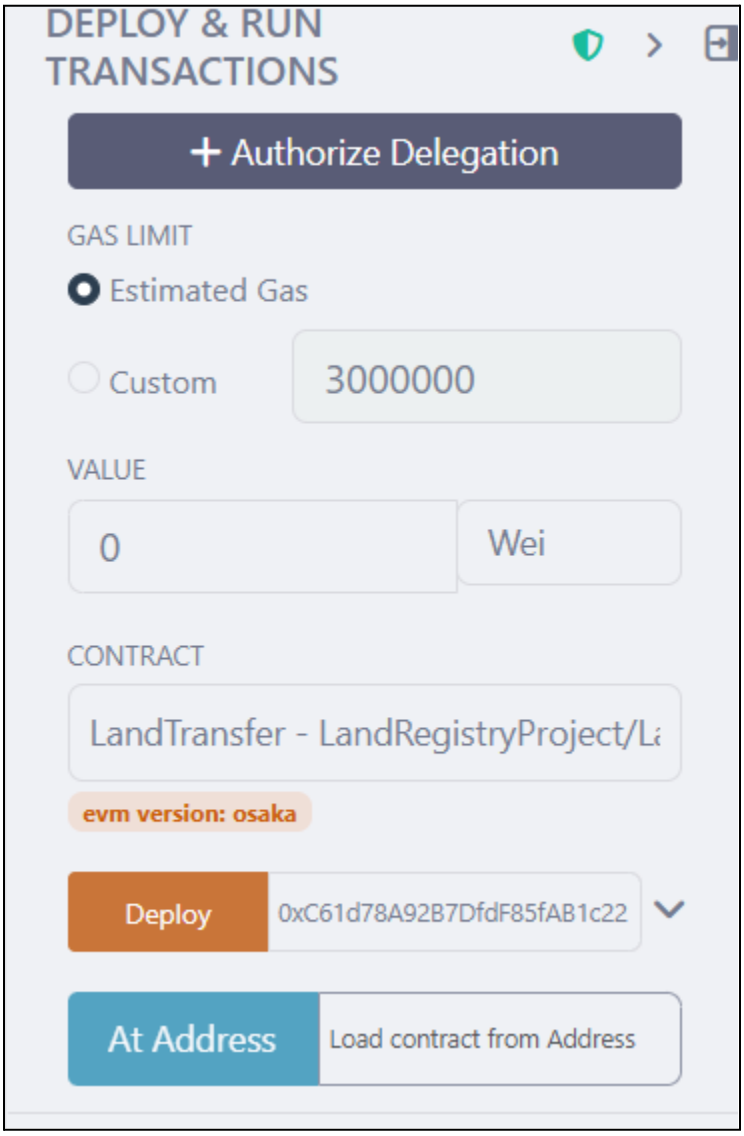
Step 4:Copy the contract address of the deployed LandRegistry contract



Step 5:Select LandTransfer here in the dropdown



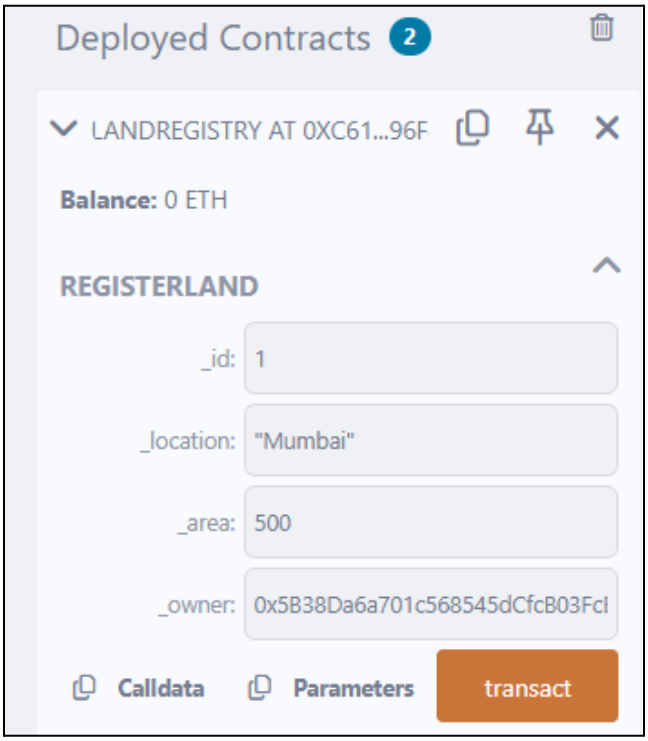
Step 6: Add the contract address in place of _registryAddress field next to the deploy button,and click on deploy so as to deploy the LandTransfer contract

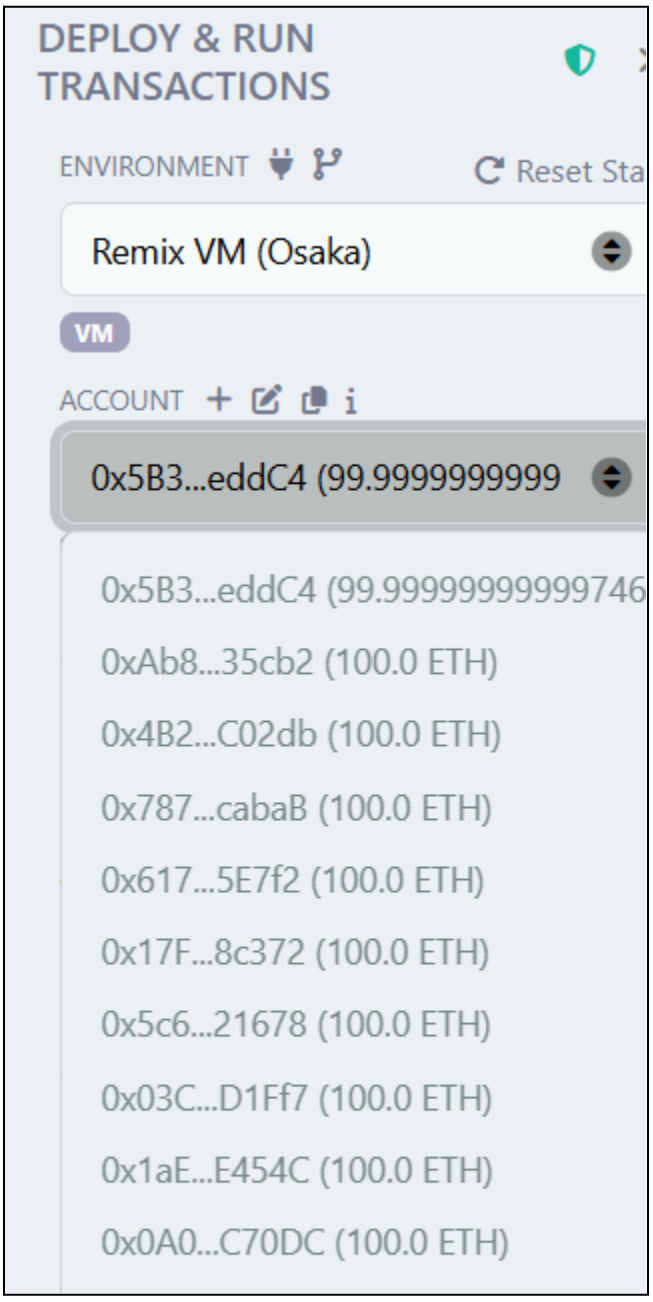


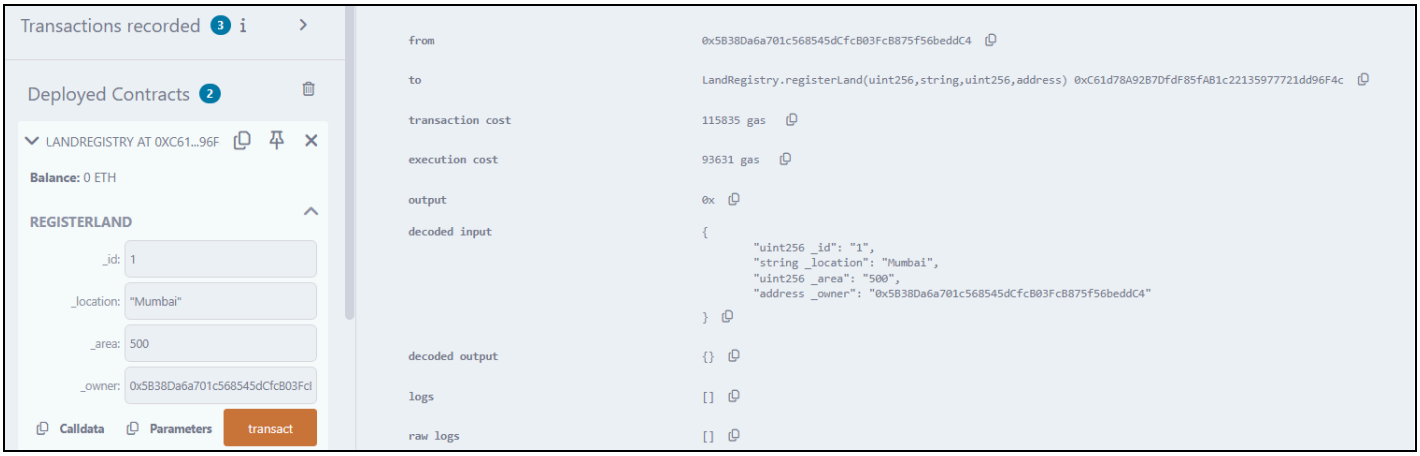
Step 7: Execute the transactions

1.Land Registration

In the deployed contracts section expand the LandRegistry contract, and add details as mentioned. Also ensure that the wallet address is added in the owner field. Once all this is done, click on transact to complete the transaction.



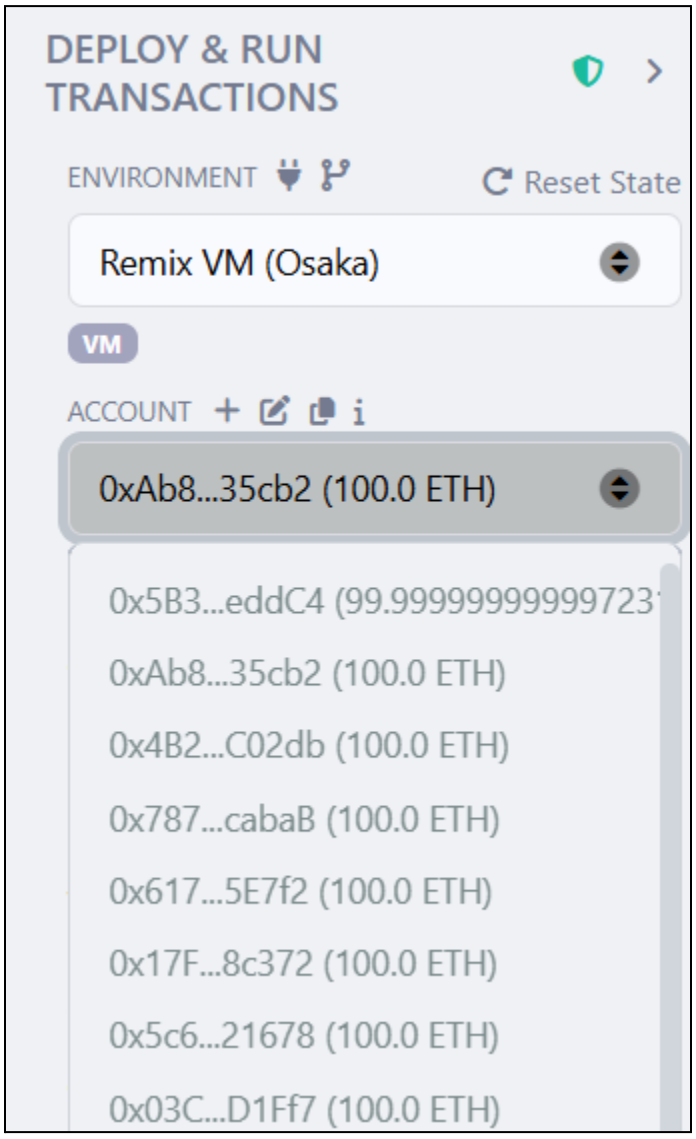

DEPLOY & RUN TRANSACTIONS
ENVIRONMENT
Reset Sta
Remix VM (Osaka)
VM
ACCOUNT
0x5B3...eddC4 (99.9999999999
0x5B3...eddC4 (99.99999999999746
0xAb8...35cb2 (100.0 ETH)
0x4B2...C02db (100.0 ETH)
0x787...cabaB (100.0 ETH)
0x617...5E7f2 (100.0 ETH)
0x17F...8c372 (100.0 ETH)
0x5c6...21678 (100.0 ETH)
0x03C...D1Ff7 (100.0 ETH)
0x1aE...E454C (100.0 ETH)
0x0A0...C70DC (100.0 ETH)



Enter id=1 in the getLand field to see the registered data

2.**Transfer Ownership**

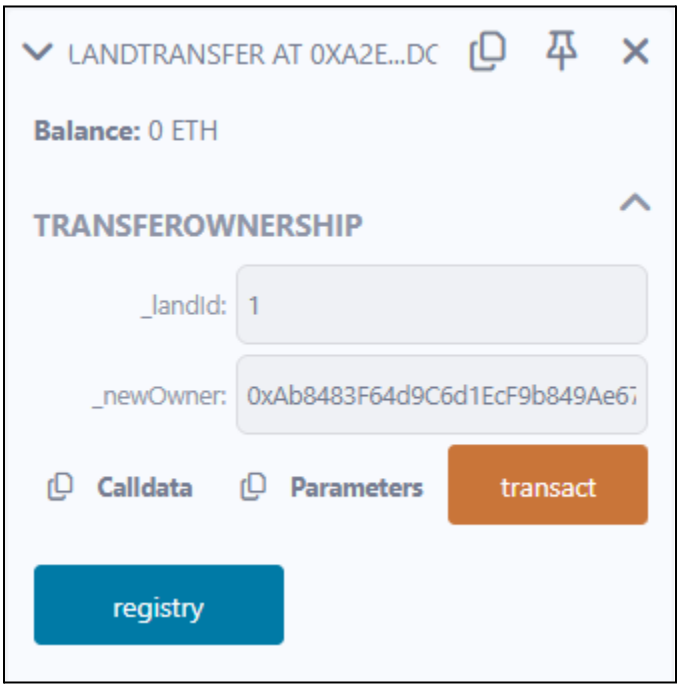
Copy a different wallet address from the account dropdown(this wallet address represents the buyer)



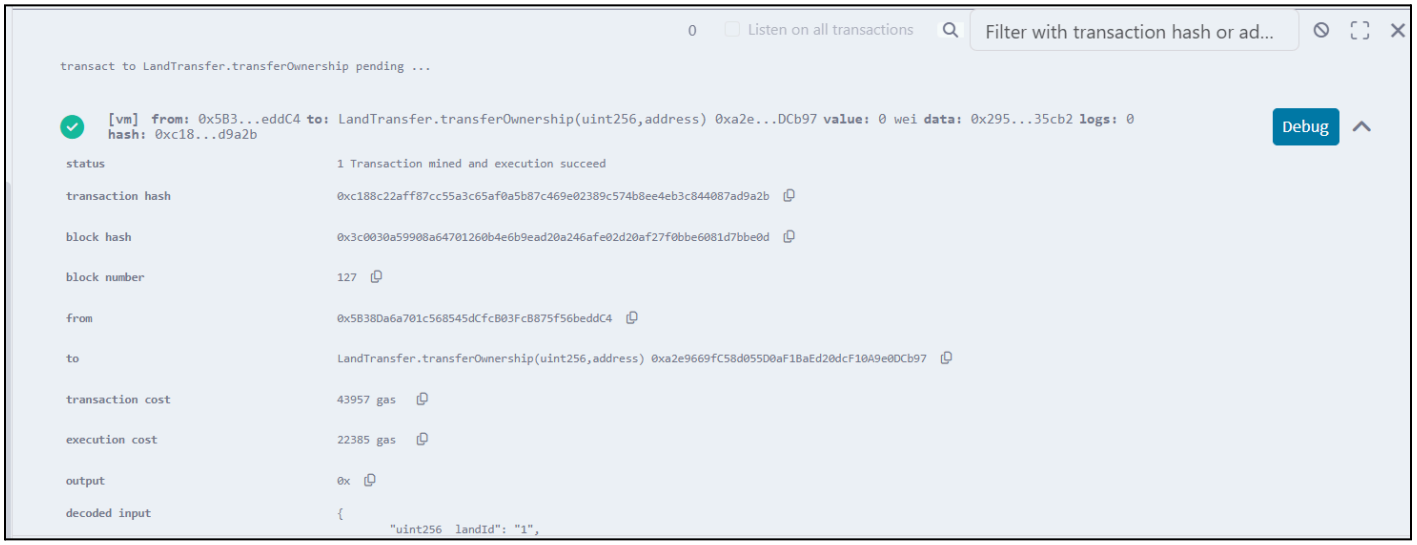
Switch back to the original wallet address(i.e the seller's wallet address) and within the transferOwnership function in the land transfer contract, enter:-

_landId: **1**

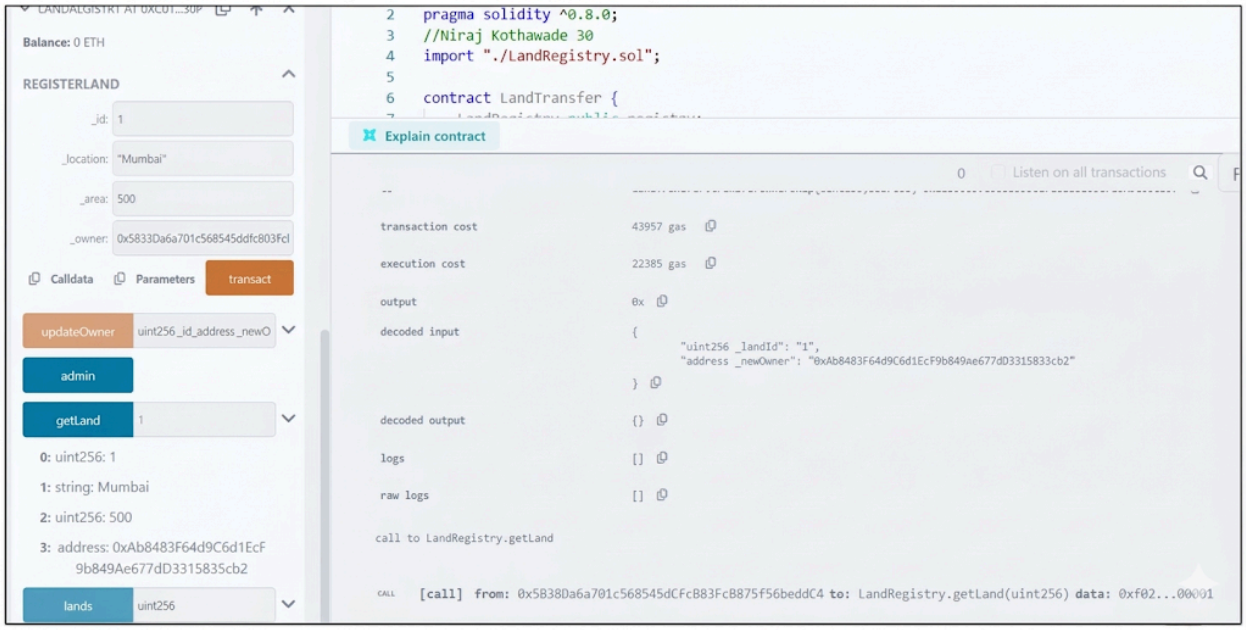
_newOwner: **[The Buyer's Address]**



Click on transact to complete the ownership transfer



Now, verify the ownership transfer by going to the LandRegistry contract. When we click on the getLand button, we see a different wallet address, indicating that the ownership transfer has taken place and is successfully verified.



Conclusion:-

This experiment helped in understanding how smart contracts automate processes and execute securely on the Ethereum blockchain. Using Solidity and Remix IDE, the development, deployment, and interaction with smart contracts become simple and efficient, forming the foundation for decentralized applications.